Assignment description

In this assignment we continue the work on the interpreter that we started with Assignment 4. We will be adding support for allocating and deallocating memory. We will be refactoring your existing solution so make sure that what works keeps working and recompile often so that nothing breaks. We are not changing existing functionality.

We will use a very simple model for memory. Memory addresses are integers and our model will map integer addresses to integer values. We will create support for allocating memory, freeing memory, as well as reading from and writing to memory.

Important: To pass this assignment you have to pass either the `Green` or the `Yellow` tests, but not both (they use different types). We recommend that you start with the `Green` ones and then refactor to use the `Yellow` ones. If you have done the yellow exercises from last week, we recommend you continue using errors and build off of that.

Before you start, take the template provided with the Assignment and add your `Eval.fs` and `State.fs` files from Assignment 5 to it. The template is very similar to last week's template, but the language in `Language.fs` has been extended to deal with memory allocation, deallocation as well as reading from and writing to memory. More precisely,

- The arithmetic expressions `aexpr` has had the constructor `MemRead` added, which allows reading from memory.
- The statements `stmnt` has had the constructors `Alloc`, `Free`, and `MemWrite` added which allocates, dealocates and writes to memory respectively.

A simple way to think about memory is just as a consecutive collection of addresses where each address stores a value. For instance, the following piece of memory stores the values `2`, `3`, and `42` at addresses `0`, `5`, and `9`,

respectively

```
 0 1 2 3 4 5 6 7 8  9 10 ...
------------------------
|2| | | |3| | | |42|  | ...
------------------------
```

To make your old code compile in order to get you started

- we only have one version of `aexprEval` now. We highly recommend you keep the one that uses `Option.bind` or `Result.bind`.
- Update `aexprEval` so that it always returns `0` for all `MemRead` expressions.
- Update `stmntEval` so that `stmntEval s st` returns `st` when `s` is `Alloc`, `Free`, or `MemWrite`.

The easiest way to do these final two points is to add a wild card character (`| _ -> <return value>`) to the end of `aexprEval`and stmntEval to handle the new cases from this assignment.

Make sure your code compiles.

Green Exercises

Create a file `State.fsi` that contains the signatures of the `state` type and the select functions from `State.fs` that you created for Assignment 5. More specifically, create signatures for

- The `state` type
- The Green or Yellow functions `mkState`, `declare`, `getVar` and `setVar`
- The Red functions `push` and `pop` if you did them

Recompile your solution. It should work right off the bat, but if it doesn't it most likely means that you are calling functions from `State.fs` in `Eval.fs` that you shouldn't be (`validVarialeName` and `reservedVariableName` for instance).

In Memory.fs

Create a type `memory` that contains your memory as

- a map from integers (memory addresses) to integers (values)

- an integer containing the next available free memory address, which we call `next`.

The intuition here is that the map maps memory addresses to values and whenever we allocate memory the `next` pointer is increased, but to keep things simple we will never decrease `next`.

Remember that you must wrap your type in a disjoint union or a record type in order for it to work with the `.fsi` file.

Create a function `empty` of type `int -> memory` that given an integer `memSize` returns a blank memory represented as the empty map and `next` set to `0`. The `memSize` argument is not used, but it must be there to be compatible with the Red exercises. But here, you must include it, but the body of the function should just ignore it.

Create a function `alloc` of type `int -> memory -> (memory * int) option` that given an amount of memory to allocate `size` and memory returns

- `Some(mem', next)` where `mem'` is identical to `mem` but where all addresses in from `next` to `next+size-1` are set to `0`, if `size` is strictly greater than `0` and where the `next` pointer is updated to `next + size`
- `None` otherwise (if `size` is smaller than or equal to `0`)

Create a function `free` of type `int -> int -> memory -> memory option` that given a memory position `ptr`, a size `size`, and memory `mem` returns

- `Some mem'` where `mem'` is identical to `mem`, but where all addresses from `ptr` to `ptr + size - 1` have been removed, as long as all of these addresses are allocated in `mem`
- `None` otherwise Note that `free` does not decrease the `next` pointer.

Create a function `setMem` of type `int -> int -> memory -> memory option` that given an address `ptr`, a value `v`, and memory `mem` returns

- `Some mem'` where `mem'` is equal to `mem` but where the the address `ptr` has been mapped to `v`, if `ptr` is allocated.
- `None` othrewise

Create a function `getMem` of type `int -> memory -> int option` that given

an address `ptr` and memory `mem` returns

- `Some v`, where `v` is the value stored in `mem` at address `ptr`, if `ptr` is allocated
- `None` otherwise

Examples:

```
> 10 |> empty |> getMem 3
- val it: int option = None

> 10 |> empty |> setMem 3 42
- val it: memory option = None

> 10 |> empty |> alloc 3 |>
  Option.bind (fun (m, ptr) -> m |> setMem ptr 42 |> Option.bind
(getMem ptr))
- val it: int option = Some 42

> 10 |> empty |> alloc 3 |>
  Option.bind (fun (m, ptr) -> m |> setMem ptr 42 |> Option.bind
(getMem (ptr + 2)))
- val it: int option = Some 0

> 10 |> empty |> alloc 3 |>
  Option.bind (fun (m, ptr) -> m |> setMem ptr 42 |> Option.bind
(getMem (ptr + 3)))
- val it: int option = None

> 10 |> empty |> alloc 3 |>
  Option.bind (fun (m, ptr) -> m |> setMem ptr 42 |>
                               Option.bind (free (ptr + 1) 2) |>
                               Option.bind (getMem ptr))
- val it: int option = Some 42



> 10 |> empty |> alloc 3 |>
  Option.bind (fun (m, ptr) -> m |> setMem ptr 42 |>
```

```
                                Option.bind (free ptr 2) |>
                                Option.bind (getMem ptr))
- val it: int option = None

> 10 |> empty |> alloc 3 |>
  Option.bind (fun (m, ptr) -> m |> setMem ptr 42 |> Option.bind
(free ptr 4))
- val it: memory option = None
```

In Memory.fsi

Add a reference to the `memory` type, but don't declare the type here as well. The only implementation should be in the `Memory.fs`.

Add the signatures for `empty`, `alloc`, `free`, `getMem`, and `setMem`.

In State.fs

Update the `state` datatype to also, in addition to the variable store from Assignment 5, include memory of type `memory`. You can either use a `record` or a discriminated union to include both.

The original `mkState` has type `unit -> state`. Change it to have type `int -> state` that given an integer `memSize` returns the same state as in Assignment 5, but with the memory set to `Memory.empty memSize`

Update `declare`, `getVar`, and `setVar` to use your new version of `state`. This should be a very short refactor. Make sure that your existing program from Assignment 5 still works.

Create a function `alloc` of type `string -> int -> state -> state option` that given a variable `x` and a size of memory to allocate `size`, and a state `st` allocates `size` cells of memory in `st` and returns

- `Some st'`, where `st'` is equal to `st` but where the memory has been allocated and where the variable `x` points to the newly allocated memory
- `None` if either memory allocation fails or if writing to the variable x fails

`create` functions `free`, `getMem`, and `setMem` that have the same signatures as in `Memory.fs` except all occurrences of `mem` have been replaced by `state`.

The `free` function, for instance, should have type `int -> int -> state -> state option`. These functions all leave the variable store alone but use their corresponding functions from `Memory.fs` to read and modify the memory part of the state.

Examples:

```
> 10 |> mkState |> getMem 0
-- val it: int option = None

> 10 |> mkState |> setMem 0 42
-- val it: state option = None

> 10 |> mkState |> alloc "x" 2
-- val it: state option = None

> 10 |> mkState |> declare "x" |>
  Option.bind (alloc "x" 2) |>
  Option.bind (fun st -> st |> getVar "x" |> Option.bind (fun ptr ->
getMem ptr st))
-- val it: int option = Some 0

> 10 |> mkState |> declare "x" |>
  Option.bind (alloc "x" 2) |>
  Option.bind (fun st -> st |> getVar "y" |> Option.bind (fun ptr ->
getMem ptr st))
-- val it: int option = None

> 10 |> mkState |> declare "x" |>
  Option.bind (alloc "x" 2) |>
  Option.bind (fun st -> st |>
                         getVar "x" |>
                         Option.bind (fun ptr -> st |>
                                                 setMem ptr 42 |>
                                                 Option.bind
(getMem ptr)))
-- val it: int option = Some 42
```

```
> 10 |> mkState |> declare "x" |>
  Option.bind (alloc "x" 3) |>
  Option.bind (fun st -> st |>
                         getVar "x" |>
                         Option.bind (fun ptr -> st |>
                                                 setMem ptr 42 |>
                                                 Option.bind (free
(ptr + 1) 2) |>
                                                 Option.bind
(getMem ptr)))
- val it: int option = Some 42

> 10 |> mkState |> declare "x" |>
  Option.bind (alloc "x" 3) |>
  Option.bind (fun st -> st |>
                         getVar "x" |>
                         Option.bind (fun ptr -> st |>
                                                 setMem ptr 42 |>
                                                 Option.bind (free
ptr 2) |>
                                                 Option.bind
(getMem ptr)))
- val it: int option = None

> 10 |> mkState |> declare "x" |>
  Option.bind (alloc "x" 3) |>
  Option.bind (fun st -> st |>
                         getVar "x" |>
                         Option.bind (fun ptr -> st |>
                                                 setMem ptr 42 |>
                                                 Option.bind (free
ptr 4)))
- val it: state option = None
```

In State.fsi

Add signatures for `alloc`, `free`, `getMem`, and `setMem`

In Eval.fs

As in Assignment 5 you may not pattern match on state in any way but must use the functions from `State.fs`.

Modify `aexprEval`, which still has type `aexpr -> state -> int option` such that given an arithmetic expression `a` and a state `st` returns

- Some v if `a` is equal to `MemRead e1`, e1 evaluates to `Some ptr` and `ptr` points to `v` in the memory of the state.
- Like `aexprEval` in Assignment 5 otherwise

Modify `stmntEval`, which still has type `stmnt -> state -> state option` so that given a statement `s` and a state `st` returns

- `Some st'` if `s` is equal to `Alloc(x, e)`, the variable `x` is declared, `e` evaluates to `Some size`, and allocating `size` cells in memory evaluates to `Some(st'', ptr)`, where `st'` is equal to `st''` but with the variable `x` set to `ptr`.
- `Some st'` if `s` is equal to `Free(e1, e2)`, `e1` evaluates to `Some ptr`, `e2` evaluates to `Some size`, and `st'` is the state `st` where `size` cells have successfully been freed from `st`, starting at the point `ptr`.
- `Some st'` if `s` is equal to `MemWrite(e1, e2)`, `e1` evaluates to `Some ptr`, `e2` evaluates to `Some v`, and `st'` is the state identical to `st` but where the pointer `ptr` has successfully been update to point to `v`.
- Like `stmntEval` in Assignment 5 otherwise

Examples:

```
> 10 |>
  mkState |>
  stmntEval (Seq (Declare "x", Seq(Alloc ("x", Num 3), Seq(Declare
"result", Assign("result", MemRead (Var "x"))))))  |>
  Option.bind (getVar "result")
- val it: int option = Some 0

>10 |>
  mkState |>
  stmntEval (Seq (Declare "x", Seq(Alloc ("x", Num 3),
```

```
Seq(MemWrite(Var·"x",·Num·42),·Seq(Declare·"result",·
Assign("result",·MemRead·(Var·"x")))))))·|>·
··Option.bind·(getVar·"result")
-··val·it:·int·option·=·Some·42

>·10·|>·
··mkState·|>·
··stmntEval·(Seq·(Declare·"x",·Seq(Alloc·("x",·Num·3),·
Seq(MemWrite(Var·"x",·Num·42),·Seq(Declare·"result",·
Assign("result",·MemRead·(Var·"x"·.+.·Num·1)))))))·|>·
··Option.bind·(getVar·"result")
val·it:·int·option·=·Some·0

>·10·|>·
··mkState·|>·
··stmntEval·(Seq·(Declare·"x",·Seq(Alloc·("x",·Num·3),·
Seq(MemWrite(Var·"x",·Num·42),·Seq(Declare·"result",·
Assign("result",·MemRead·(Var·"x"·.+.·Num·3)))))))·|>·
··Option.bind·(getVar·"result")
val·it:·int·option·=·None

>·10·|>·
··mkState·|>·
··stmntEval·(Seq·(Declare·"x",·Seq(Alloc·("x",·Num·3),·
Seq(MemWrite(Var·"x",·Num·42),·Seq(Free(Var·"x"·.+.·Num·1,·Num·2),·
Seq(Declare·"result",·Assign("result",·MemRead·(Var·"x")))))))·|>·
··Option.bind·(getVar·"result")
val·it:·int·option·=·Some·42

>·10·|>·
··mkState·|>·
··stmntEval·(Seq·(Declare·"x",·Seq(Alloc·("x",·Num·3),·
Seq(MemWrite(Var·"x",·Num·42),·Seq(Free(Var·"x",·Num·2),·
Seq(Declare·"result",·Assign("result",·MemRead·(Var·"x")))))))·|>·
··Option.bind·(getVar·"result")
val·it:·int·option·=·None

>·10·|>·
```

```
··mkState·|>·
··stmntEval·(Seq·(Declare·"x",·Seq(Alloc·("x",·Num·3),·
Seq(MemWrite(Var·"x",·Num·42),·Seq(Free(Var·"x",·Num·4))))))·|>·
··Option.bind·(getVar·"result")
val·it:·int·option·=·None

>·10·|>·
··mkState·|>·
··stmntEval·(Seq·(Declare·"x",·Seq(Alloc·("x",·Num·3),·
Seq(MemWrite(Var·"x",·Num·42),·Free(Var·"x",·Num·4))))))·|>·
··Option.bind·(getVar·"result")
val·it:·int·option·=·None
```

Yellow exercises

This exercise extends the yellow exercise from Assignment 5. We're going to add proper error messages to the memory management functions.

The `Language.fs` has been updated to include the following errors

- `OutOfMemory`, which is used if you run out of memory, which in turn is only relevant if you do the red exercises.
- `NegativeMemoryAllocated` which is used if you try to allocate a memory chunk of size smaller than or equal to zero.
- `MemoryNotAllocated` which is used if you try to read from or write to memory that has not been allocated.

In Memory.fs

Refactor the code from the Green exercises to return a `Result<<result type>, Error>` rather than an `<result type> option` such that they return `Ok result` in stead of `Some result` when the functions succeed and return an `Error`, rather than `None`, such that

- `alloc memSize mem` returns the error
  - `OutOfMemory` if there is not `memSize` memory available in `mem` (only relevant if you do the red exercises)
  - `NegativeMemoryAllocated(memSize)` if `memSize` is smaller than or equal to zero

- `free ptr size mem` returns the error `MemoryNotAllocated(ptr')` if any memory in `mem` between `ptr` and `ptr + size - 1` is not allocated, where `ptr'` is the smallest address greater than or equal to `ptr` that is not allocated.
- `setMem ptr v mem` and `getMem ptr mem` returns `MemoryNotAllocated(ptr)` if `ptr` is not allocated.

In State.fs

Update `alloc`, `free`, `setMem` and `getMem` to return the same errors as in `Memory.fs`

In Memory.fsi and State.fsi

Update the signatures to match the new return types.

In Eval.fs

Update `aexprEval`, `bexprEval`, and `stmntEval` to return `Result` types rather than `Option` types. This should not require making any major changes from Assignment 5, providing you have compartemantalised everything properly in `State.fs` and `Memory.fs`

Red exercises

Here you will make changes to `Memory.fs` only to make more optimal functions for memory management. The signatures in `Memory.fsi` should not change and your optimisations here should not affect the rest of the project in any way. This also means that this assignment is a bit open ended. If you think you have a better way of managing memory than presented here, go for it.

Moreover, you should only have to make changes to `Memory.fs`. All other files should stay exactly the same.

Have your `memory` type contain

- A fixed-size array containing the cells in the memory. The size will be set on memory creation (when calling `empty`).
- A Map of type `Map<int, int list>`, which we call the free map, where the key is the size of an available memory block, and every integer in the

value list represents the start of a piece of unallocated memory of that size. We call each such consecutive piece of free memory a *chunk*.

For instance, the following memory of size ten

```
 0 1 2 3 4 5 6 7 8 9
--------------------
|X|X| |5|X|X|7|X|X|X|
--------------------
```

where X means memory that has not been allocated, would be represented by an array `arr` where `arr.[3] = 5` and `arr.[6] = 7` (and the rest of the array can be absolutely anything), and the free map would be represented by `map [(2, [0; 4]); (3, [7])]` meaning that there are two free chunks of size `2` starting at indexes `0` and `4` respectively, and one free chunk of size `3` starting at index `7`.

Defragmentation

When allocating and deallocating memory it may happen that adjacent chunks are put in the free map.

Create a function `defrag` of type `memory -> memory` that given memory `mem` returns exactly the same memory, but where any adjacent chunks in the free map have been merged.

The main functions

Modify the functions in `Memory.fs` such that error handling is managed as defined by the Green or Yellow exercises, but otherwise work as follows:

- `empty memSize` returns an empty memory with an array of size `memSize` and the free chunk map set to `map [(memSize, [0])]`, the intuition being that it contains one big free chunk of size `memSize`.
- `alloc size mem` allocates `size` memory in `mem` by
  - taking the smallest chunk of size `freeSize` from the free map, where `freeSize` is greater than or equal to `size`, and removing it from `mem`
  - put the remaining free memory of size `freeSize - size` back into the free map (assuming the size is greater than zero). Note that there

    may already be chunks of this size in the free map, in which case you just add it to the start of the chunk list for chunks of that size.
    - If there is not enough free memory, then first try to defragment the memory and allocate again and if that fails then return either `None`, if doing the green exercises, or the error `OutOfMemory` if doing the yellow.
- `free ptr size mem` frees memory starting at `ptr` and ending in `ptr + size - 1`. You do not have to defragment the memory here.
- `setMem ptr v mem` sets the index `ptr` of the memory array to `v` , assuming `ptr` is allocated.
- `getMem ptr mem` gets the value stored at index `ptr` in the memory array, assuming `ptr` is allocated.

Determining wether or not a piece of memory is allocated or not is non-trivial in this setup. By far the easiest is to use `Map.exists` on your free map.